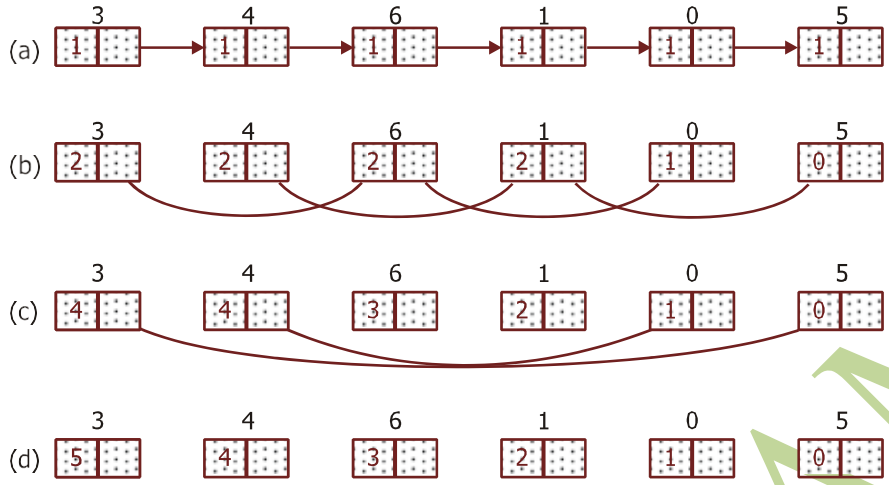
BÀI 2: DANH SÁCH



Mục tiêu
Sau khi học bài này, các bạn có thể: • Nắm được khái niệm về danh sách và danh sách liên kết, phân biệt được các dạng của danh sách liên kết. • Nắm được các thao tác trên danh sách liên kết. • Nắm được cách cài đặt danh sách bằng mảng và bằng danh sách liên kết. • Nắm được cách cài đặt các thao tác trên danh sách và danh sách liên kết đơn. • Vận dụng cấu trúc dữ liệu của danh sách vào giải quyết các bài toán thực tế.

Nội dung
• Khái niệm danh sách và các thao tác trên danh sách. • Biểu diễn danh sách bằng mảng. • Danh sách liên kết. • Biểu diễn danh sách liên kết bằng cấu trúc mảng. • Ứng dụng danh sách.

Thời lượng học
6 tiết

AUMVIETNAM

Trong lập trình, các danh sách là các kiểu dữ liệu trừu tượng rất hữu dụng. Ở một số ngôn ngữ lập trình danh sách có thể là cấu trúc dữ liệu tiền định. Như trong ngôn ngữ lập trình danh sách (LISP), danh sách là cấu trúc dữ liệu chính được cung cấp trong LISP hay trong C++ thì danh sách đã được xây dựng sẵn và được cung cấp trong thư viện chuẩn. Tuy nhiên trong đa số các ngôn ngữ lập trình thì danh sách không phải là cấu trúc dữ liệu tiền định do đó các kỹ thuật để xây dựng cấu trúc danh sách là rất quan trọng. Trong bài này, chúng ta sẽ xem xét một cách chi tiết các danh sách tổng quát và cách xây dựng danh sách dựa trên nền tảng cấu trúc dữ liệu là mảng và danh sách liên kết.

Trong công việc hàng ngày, chúng ta sử dụng các danh sách thường xuyên như danh sách học viên của một lớp, danh sách những người đăng kí mua vé máy bay, danh sách những người đang chờ khám bệnh, danh sách các cuộc triển lãm đã được tổ chức vào năm 2004 tại Hà Nội…v.v. Trong mỗi danh sách này đều lưu trữ các thông tin mà chúng ta cần quan tâm và có thể là phải nhớ để phục vụ cho công việc của chúng ta.

Xét danh sách học viên của một lớp danh sách này ghi thông tin của tất cả các học viên trong lớp như: họ tên, ngày tháng năm sinh, quê quán, giới tính, chỗ ở hiện nay… nhằm phục vụ trong công tác quản lí học viên; thông tin về mỗi học viên trong lớp là một phần tử của danh sách. Do số học viên trong một lớp là hữu hạn nên danh sách này có số phần tử hữu hạn. Thông tin về các học viên trong danh sách thường được sắp xếp theo một thứ tự nào đó (thường sắp xếp theo thứ tự từ điển về tên và họ) để tiện cho việc quản lý. Khi đó nếu có thêm học viên nào vào lớp hay có học viên chuyển sang lớp khác thì danh sách học viên này phải thay đổi bằng cách bổ sung hay xóa bớt đi thông tin về học viên. Cách giải quyết các bài toán này như thế nào sẽ được trình bầy trong bài này.

2.1. Khái niệm danh sách và các thao tác trên danh sách

2.1.1. Khái niệm

> Danh sách là một cấu trúc dữ liệu gồm một hữu hạn các phần tử có kiểu dữ liệu xác định và giữa các phần tử có mối liên hệ với nhau.
>
> Trên phương diện **toán học**, **danh sách** là tập hợp hữu hạn các phần tử, có thứ tự và giữa chúng có mối liên hệ tuyến tính.

Mối liên hệ tuyến tính đó là mỗi phần tử trong danh sách trừ phần tử đầu tiên có duy nhất một phần tử đứng trước nó và mỗi phần tử trong danh sách trừ phần tử cuối cùng có duy nhất một phần tử đứng sau nó.

Giả sử L là danh sách gồm một dãy n phần tử được biểu diễn như sau:

$$L = (a_1, a_2, ..., a_n).$$

Ở đây $n > 0$ và mỗi a_i là một phần tử $(1 \le i \le n)$

Ta gọi số n là ***độ dài*** của của danh sách. Như vậy độ dài của danh sách là số phần tử của danh sách đó. Nếu $n \ge 1$ thì a_1 được gọi là ***phần tử đầu tiên*** của danh sách, còn a_n là ***phần tử cuối cùng*** của danh sách. Nếu $n = 0$ tức là danh sách không có phần tử nào, thì danh sách được gọi là ***rỗng***.

Một tính chất quan trọng của danh sách là các phần tử của nó được sắp tuyến tính: a_i (i = 1, 2, ..., n) là phần tử ở vị trí thứ i của danh sách thì phần tử a_i "đi trước" phần tử a_{n+1} và "đi sau" phần tử a_{n-1}.

Ví dụ 2.1.

Tập hợp họ tên các sinh viên của lớp TINHOC 28 được liệt kê trên giấy như sau:

1. Nguyễn Trung Cang
2. Nguyễn Ngọc Chương
3. Lê Thị Lệ Sương
4. Trịnh Vũ Thành
5. Nguyễn Phú Vĩnh

là một danh sách. Danh sách này gồm có 5 phần tử, mỗi phần tử có một vị trí trong danh sách theo thứ tự xuất hiện của nó.

Danh sách con

Nếu L = (a_1, a_2, ..., a_n) là một danh sách và i, j là các vị trí, $1 \le i \le j \le n$ thì danh sách L' gồm tất cả các phần tử từ a_i đến a_j của danh sách L được gọi là danh sách con của L. Danh sách rỗng được xem là danh sách con của một danh sách bất kỳ.

Danh sách con bất kỳ gồm các phần tử bắt đầu từ phần tử đầu tiên của danh sách L được gọi là *phần đầu* của danh sách L. *Phần cuối* của danh sách L là một danh sách con bất kỳ kết thúc ở phần tử cuối cùng của danh sách L.

Dãy con

Một danh sách được tạo thành bằng cách loại bỏ một số (có thể bằng không) phần tử của danh sách L được gọi là *dãy con* của danh sách L.

Ví dụ 2.2.

Xét danh sách gồm 6 số nguyên dương

```
L = (1, 2, 3, 4, 5, 6)
```

Khi đó danh sách `(2, 3, 4)` là danh sách con của `L`. Danh sách `(3, 4, 5, 6)` là dãy con của `L`. Danh sách `(1, 2, 3)` là phần đầu, còn danh sách `(4, 5, 6)` là phần cuối của danh sách `L`.

2.1.2. Các thao tác trên danh sách

Tùy thuộc vào đặc điểm và tính chất của từng loại danh sách mà mỗi danh sách có thể có hoặc chỉ cần có một số thao tác nhất định nào đó. Mặc dù các thao tác của danh sách có thể thay đổi theo ứng dụng thì chúng thường bao gồm các thao tác sau:

- **Tạo mới một danh sách**

  Trong thao tác này, chúng ta sẽ đưa vào danh sách nội dung của các phần tử, do vậy chiều dài của danh sách sẽ được xác định. Trong một số trường hợp, chúng ta chỉ cần khởi tạo giá trị và trạng thái ban đầu cho danh sách.

- **Thêm một phần tử vào danh sách**

  Thao tác này nhằm thêm một phần tử vào trong danh sách, nếu việc thêm thành công thì chiều dài danh sách sẽ tăng lên 1. Cũng tùy thuộc vào từng loại danh sách

AUM VIỆT NAM

và từng trường hợp cụ thể mà việc thêm phần tử sẽ được tiến hành đầu, cuối hay giữa danh sách.

- **Loại bỏ bớt một phần tử ra khỏi danh sách**

  Ngược với thao tác thêm, thao tác này sẽ loại bớt một phần tử ra khỏi danh sách do vậy nếu việc loại bỏ này thành công thì chiều dài của danh sách sẽ bị giảm đi 1. Thông thường, trước khi xóa bỏ một phần tử chúng ta phải thực hiện thao tác tìm kiếm phần tử cần loại bỏ này.

- **Tìm kiếm một phần tử trong danh sách**

  Thao tác này sẽ vận dụng các thuật toán tìm kiếm để tìm kiếm một phần tử trên danh sách thỏa mãn một tiêu chuẩn nào đó (thường là tiêu chuẩn về giá trị).

- **Cập nhật (sửa đổi) giá trị cho một phần tử trong danh sách**

  Thao tác này nhằm thay đổi nội dung của một phân tử trong danh sách. Tương tự như thao tác loại bỏ, trước khi thay đổi thường chúng ta cũng phải thực hiện thao tác tìm kiếm phần tử cần được thay đổi.

- **Sắp xếp thứ tự các phần tử trong danh sách**

  Trong thao tác này chúng ta sẽ vận dụng các thuật toán sắp xếp để sắp các phần tử trong danh sách theo một trật tự nào đó.

- **Tách một danh sách thành nhiều danh sách**

  Thao tác này thực hiện việc chia một danh sách thành nhiều danh sách con theo một qui định nào đó. Kết quả sau khi chia là tổng chiều dài trong các danh sách con phải bằng chiều dài của danh sách ban đầu.

- **Nhập nhiều danh sách thành một danh sách**

  Ngược với thao tác chia, thao tác này tiến hành nhập nhiều danh sách con thành một danh sách có chiều dài bằng tổng chiều dài các danh sách con. Tùy thuộc vào từng trường hợp yêu cầu cụ thể mà việc nhập có thể là ghép nối đuôi các danh sách lại với nhau hoặc trộn lẫn các phần tử trong các danh sách con vào danh sách lớn theo một trật tự nhất định.

- **Sao chép một danh sách**

  Thao tác này thực hiện việc sao chép toàn bộ nội dung từ một danh sách này sang một danh sách khác sao cho sau khi sao chép ta có các danh sách giống hệt nhau về nội dung.

- **Hủy danh sách**

  Thao tác này thực hiện việc xóa bỏ toàn bộ nội dung của một danh sách để thành một danh sách rỗng. Tùy thuộc vào từng loại danh sách và việc xóa bỏ này có thể bao gồm việc xóa cả nội dung và bộ nhớ hay không.

Ví dụ 2.3.

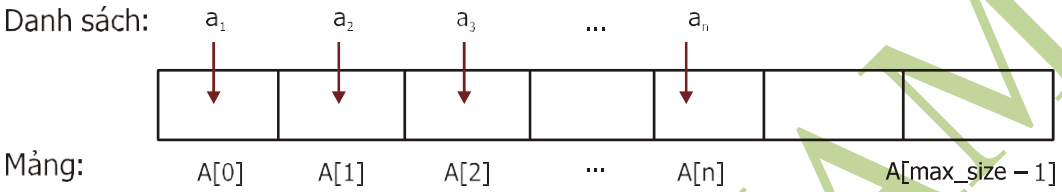
Giả sử ta có danh sách `L = (3, 2, 1, 5)`.

Ta có hàm: `ListType Delete_E(int Pos,ListType List)` để xóa một phần tử khỏi danh sách và hàm `ListType Insert_E(int Pos,E_Type Item, ListType List)` để thêm một phần tử vào danh sách. Khi đó, thực hiện `L = Delete_E(3,L)` ta được danh sách `L = (3,2,5)`. Kết quả của `L = Insert_E(1,6,L)` là danh sách `L = (6,3,2,1,5)`.

AUM VIỆT NAM

2.2. Biểu diễn danh sách bằng mảng

Cài đặt một danh sách trên máy tính là tìm một cấu trúc dữ liệu cụ thể mà máy tính có thể hiểu để lưu trữ các phần tử của danh sách và đồng thời để viết các chương trình con thực hiện các thao tác trên danh sách. Phương pháp tự nhiên nhất để cài đặt một danh sách là sử dụng mảng, trong đó mỗi thành phần của mảng sẽ lưu giữ một phần tử nào đó của danh sách, và các phần tử kế nhau của danh sách được lưu giữ trong các thành phần kế nhau của mảng. Việc sử dụng mảng để biểu diễn danh sách được thể hiện bằng hình dưới đây:



Như vậy để biểu diễn danh sách bằng mảng chúng ta có các khai báo sau :

```
#define Max_Size N
// N là độ dài cực đại của danh sách, N là một số hữu hạn
typedef  Kieu_du_lieu E_Type;
       //Đặt lại tên kiểu dữ liệu của các phần tử trong mảng
struct ListType //tên kiểu cấu trúc do người lập trình đặt
{
         E_Type Element[Max_Size];
       //mảng chứa các phần tử của danh sách
         int Size;
       //biến đếm số phần tử trong danh sách
} List;
//khai báo List có kiểu cấu trúc vừa định nghĩa là ListType
```

Khi cài đặt danh sách qua mảng như trên, các thao tác trên danh sách được thực hiện rất dễ dàng. Để khởi tạo một danh sách rỗng, chỉ gần một lệnh gán :

```
List.Size:= 0;
```

Và độ dài của danh sách là `List.Size`. Danh sách đầy, nếu `List.Size = Max_Size`. Việc duyệt danh sách cũng được thực hiện dễ ràng bằng cách sử dụng vòng lặp với chỉ số của mảng thay đổi. Ví dụ để hiển thị tất cả các phần tử của danh sách ta chỉ cần sử dụng vòng lặp:

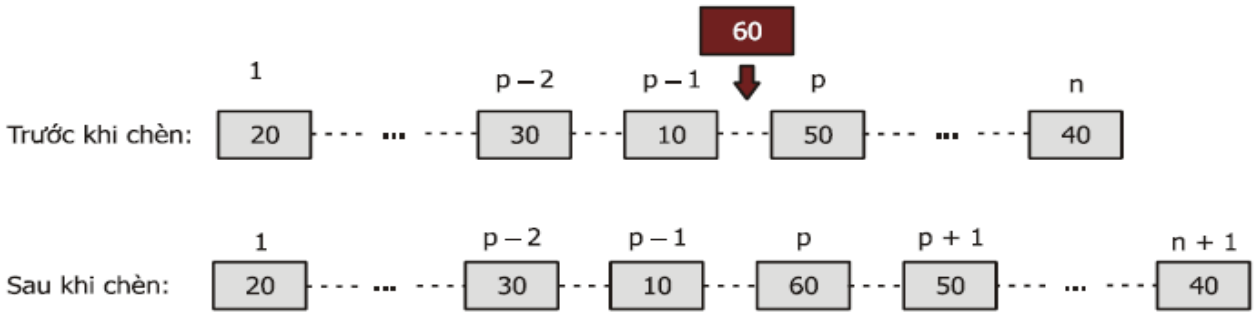
```
for (int i=0; i<List.Size; i++)
printf ("\n phan tu thu %d la : %.5f\n", i, List.Element[i]);
```

Tuy nhiên có một số thao tác trên danh sách là tương đối phức tạp như thao tác chèn một phần tử vào danh sách.

Ví dụ 2.4

Chèn thêm một phần tử có giá trị 60 vào vị trí sau phần tử có giá trị 10 của danh sách.

AUM VIỆT NAM



Theo định nghĩa, danh sách là cấu trúc dữ liệu không đưa ra giới hạn về chiều dài của danh sách nên về mặt lý thuyết thao tác chèn luôn thực hiện được. Tuy nhiên trong cách cài đặt danh sách bằng mảng này, độ dài của mảng là cố định nên làm giới hạn chiều dài của danh sách. Vì vậy trước khi chèn một phần tử mới vào danh sách phải kiểm tra xem trong mảng còn chỗ cho nó không.

Điều phức tạp thứ hai xuất phát từ việc cài đặt mảng các phần tử của danh sách được lưu trữ ở các vị trí liên tiếp trên mảng. Nên để chèn một phần tử mới vào, ta cần dịch chuyển các phần tử của mảng để tạo chỗ cho phần tử cần chèn. Ví dụ: để thực hiện thao tác chèn như trên ta phải dịch chuyển các phần tử từ thứ p đến n của mảng sang vị trí mới lần lượt từ thứ p + 1 đến thứ n + 1. sau đó mới chèn 60 vào vị trí thứ p.a

Hàm thực hiện thao tác thêm một phần tử mới vào danh sách:

```
ListType Insert_E(int Pos,E_Type Item,ListType List)
  {
          int k,j;
          if (List.Size == Max_Size)
                printf("danh sach day khong the chen them");
          else
              {
                for (k = List.Size-1; k >= Pos-2; k--)
                    {
                          j = k + 1;
                          if (k == Pos-2)
                               List.Element[j]=Item;
                          else
                               List.Element[j]=List.Element[k];
                    }
                List.Size = List.Size + 1;
              }
          return List;
  }
```

Nếu n là độ dài của danh sách; dễ dàng thấy rằng, thủ tục chèn được thực hiện trong thời gian (O(n): độ phức tạp tuyến tính là số phép tính\thời gian chạm\dung lượng bộ nhớ có xu hướng tỷ lệ thuận với độ lớn đầu vào).

Việc cài đặt thao tác xóa cũng đòi hỏi phải dịch chuyển các phần tử của mảng nếu các phần tử trong danh sách được lưu trữ trên các vị trí liên tiếp của mảng.

Hàm thực hiện thao tác loại bỏ bớt một phần tử ra khỏi danh sách:

AUM VIỆT NAM

```
// ham loai bo mot phan tu
  ListType Delete_E(int Pos,ListType List)
  {
          int k;
          if (List.Size == 0)
                printf("danh sach rong");
          else
                {
                        for (k = Pos-1; k < List.Size-1; k++)
                              List.Element[k] = List.Element[k + 1];
                         List.Size = List.Size-1;
                }
        return List;
  }
```

Và khi đó thời gian thực hiện hàm này cũng là O(n).

Việc tìm kiếm trong danh sách là một phép toán được sử dụng thường xuyên trong các ứng dụng. Chúng ta sẽ xét riêng phép toán này trong phần tìm hiểu về các thuật toán tìm kiếm sau.

Chúng ta đã cài đặt danh sách bởi mảng, tức là dùng mảng để lưu giữ các phần tử của danh sách. Do tính chất của mảng, phương pháp này cho phép ta truy cập trực tiếp đến phần tử ở vị trí bất kỳ trong danh sách một cách dễ dàng vì chúng ta chỉ cần sử dụng chỉ số để định vị vị trí các phần tử trong danh sách. Tuy nhiên phương pháp này không thuận tiện để thực hiện thao tác thêm vào và loại bỏ một số phần tử của danh sách. Như đã chỉ ra ở trên, mỗi lần chèn phần tử mới vào danh sách ở vị trí p (hoặc loại bỏ phần tử ở vị trí p) ta phải đẩy xuống dưới (hoặc lên trên) một vị trí tất cả các phần tử đi sau phần tử thứ p. Mà không gian nhớ để lưu giữ các phần tử của danh sách là cố định và bị quy định bởi cỡ của mảng. Do đó danh sách không thể phát triển quá cỡ của mảng, phép toán chèn vào sẽ không được thực hiện khi mảng đã đầy. Trong những đòi hỏi chèn và xóa ở một mục bất kỳ trong danh sách, cách cài đặt tốt nhất là dùng danh sách liên kết sẽ được mô tả ở phần tiếp theo.

2.3. Danh sách liên kết

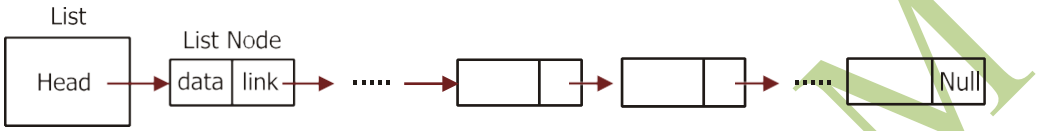
Danh sách là một dãy các mục dữ liệu nên có một thứ tự gắn liền với các phần tử trong danh sách; nó có phần tử đầu tiên, phần tử thứ hai… Như vậy bất kỳ một cách cài đặt nào của cấu trúc dữ liệu này cũng phải dùng một phương pháp nào đó để biểu diễn thứ tự này. Trong cách cài đặt danh sách bằng mảng, thứ tự các phần tử của danh sách được biểu diễn dưới dạng **ẩn** bằng cách dùng thứ tự của các phần tử mảng. Trong phần này chúng ta sẽ xem xét một cách cài đặt khác của danh sách trong đó thứ tự các phần tử của danh sách được biểu diễn dưới dạng **hiển**.

AUM VIỆT NAM

Khái niệm:

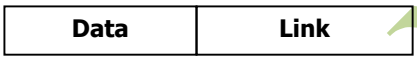
Danh sách liên kết là tập hợp các phần tử gọi là nút (node), mỗi nút chứa dữ liệu về phần tử của danh sách và con trỏ chỉ vị trí của nút chứa phần tử khác trong danh sách. Con trỏ là địa chỉ trong bộ nhớ về nút tiếp theo của danh sách. Do vậy các nút trong danh sách không cần phải lưu trữ liên tiếp nhau trong bộ nhớ và kích thước của danh sách có thể mở rộng tùy ý (chỉ giới hạn về dung lượng bộ nhớ).

Hình dưới đây thể hiện cấu trúc của một danh sách liên kết:



Head: Con trỏ trỏ tới nút (phần tử) đầu tiên của danh sách liên kết.

Như vậy cấu tạo của một nút trong danh sách liên kết có thể hình dung như sau:



Trong đó:

- phần **Data**: chứa thông tin về phần tử của danh sách
- phần **Link**: chứa địa chỉ về một nút khác trong danh sách

Riêng nút cuối cùng vì không có nút tiếp theo nữa nên trường LINK của nó phải chứa 1 "địa chỉ đặc biệt", chỉ mang tính chất quy ước, dùng để đánh dấu nút kết thúc danh sách chứ không chứa các địa chỉ ở các nút khác, ta gọi nó là "địa chỉ null" hay "con trỏ rỗng" (null pointer).

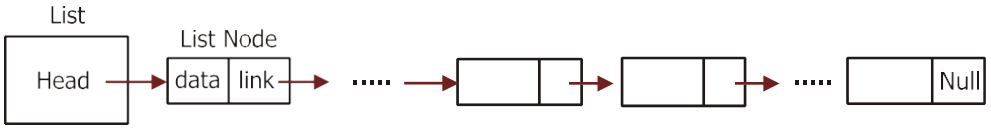
Ví dụ 2.5.

Ta có một danh sách tên các sinh viên bất kỳ.

Bây giờ ta muốn đưa ra danh sách theo thứ tự "từ điển" ta có thể tổ chức móc nối như sau:

STT	Data	Link
1	Công	4
2	Hiệp	0
3	Anh	1
4	Đồng	2

Một danh sách liên kết chứa các tên sinh viên theo thứ tự "từ điển" có thể được biểu diễn như sau:



AUM VIỆT NAM

Trong sơ đồ này các mũi tên thể hiện mỗi liên kết giữa các nút và List chỉ vào nút đầu tiên trong danh sách. Nút đầu tiên có địa chỉ là 3 (chính là số thứ tự của tên sinh viên trong danh sách thực), dựa vào trường LINK ta biết được nút tiếp theo có địa chỉ là 1, sau nút 1 là nút 4, sau nút 4 là nút 2, sau nút 2 không còn nút nào: 2 là nút kết thúc danh sách nên không có mũi tên đi ra và chỉ ra rằng phần tử này không có phần tử tiếp theo.

Chú ý

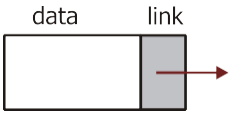
Không nên nhầm lẫn danh sách và danh sách liên kết. Danh sách và danh sách liên kết là hai khái niệm hoàn toàn khác nhau. Danh sách là một mô hình dữ liệu, nó có thể được cài đặt bởi các cấu trúc dữ liệu khác nhau. Còn danh sách liên kết là một cấu trúc dữ liệu, ở đây nó được sử dụng để biểu diễn danh sách. Do vậy các thao tác trên danh sách đều có thể thực hiện được trên danh sách liên kết.

Khi dùng danh sách liên kết để biểu diễn danh sách, tùy thuộc vào mức độ và cách thức kết nối giữa các nút trong danh sách liên kết mà danh sách liên kết có thể chia thành nhiều loại:

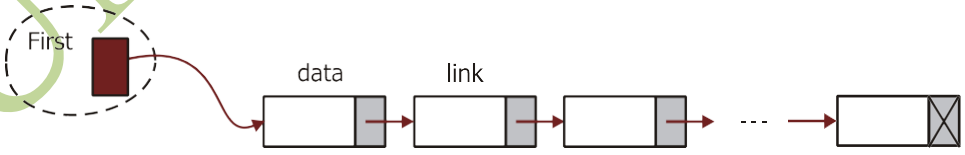
- **Danh sách liên kết đơn**

  Danh sách liên kết đơn là danh sách liên kết bao gồm các nút được nối với nhau theo một chiều. Mỗi nút gồm hai gồm hai thành phần:
  - Thành phần thứ nhất chứa giá trị lưu trong nút đó.
  - Thành phần thứ hai chứa con trỏ (địa chỉ) tới nút kế tiếp trong danh sách, nghĩa là chứa thông tin đủ để nhận biết nút kế tiếp trong danh sách là nút nào. Trong trường hợp là nút cuối cùng, trường liên kết được gán một giá trị đặc biệt (gọi là con trỏ rỗng **null**).

Cấu tạo của nút trong liên kết đơn được thể hiện trong hình dưới đây:



Và cấu trúc của danh sách liên kết đơn được thể hiện qua hình sau:



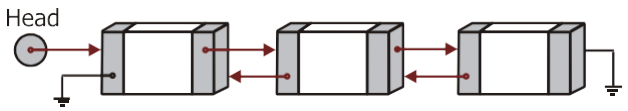
- **Danh sách liên kết đôi/ kép**

  Danh sách liên kết kép là danh sách liên kết mà mỗi nút trong danh sách ngoài thành phần dữ liệu còn có hai con trỏ, một trỏ tới nút kế tiếp, một trở đến nút liền trước nó. Như vậy trong danh sách liên kết kép mỗi nút gồm ba thành phần:
  - Thành phần thứ nhất chứa giá trị lưu trong nút đó.
  - Thành phần thứ hai là con trỏ chứa liên kết tới nút kế tiếp (con trỏ Link), nghĩa là chứa thông tin đủ để nhận biết nút kế tiếp trong danh sách là nút nào. Trong trường hợp là nút cuối cùng, trường liên kết được gán một giá trị đặc biệt (gọi là con trỏ rỗng **null**).

AUM VIET NAM

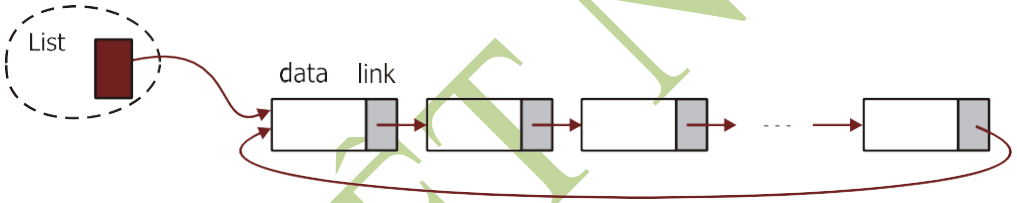
- Thành phần thứ ba là con trỏ chứa liên kết tới nút liền trước (Con trỏ Prev), nghĩa là chứa thông tin đủ để nhận biết nút liền trước nút đó trong danh sách là nút nào. Trong trường hợp là nút đầu tiên, trường liên kết được gán một giá trị đặc biệt (gọi là con trỏ rỗng **null**).

Cấu trúc của danh sách liên kết kép được thể hiện ở hình dưới đây:



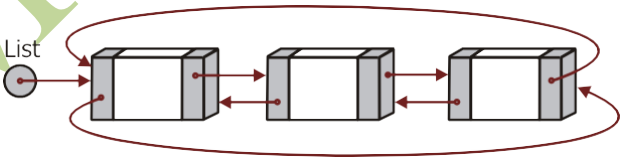
- **Danh sách liên kết vòng đơn**

  Trong danh sách liên kết đơn, trường liên kết của nút cuối cùng được gán một giá trị đặc biệt. Nếu ta cho trường liên kết của nút cuối cùng trỏ thẳng về nút đầu tiên của danh sách thì khi đó ta sẽ được một kiểu danh sách liên kết mới gọi là danh sách liên kết vòng đơn. Cấu trúc của danh sách liên kết vòng đơn được mô tả bằng hình dưới đây.



- **Danh sách liên kết vòng kép**

  Danh sách liên kết vòng kép là danh sách liên liên kết kép trong trường hợp con trỏ Prev của nút đầu tiên trong danh sách trỏ thẳng đến nút cuối cùng trong danh sách và con trỏ Link của nút cuối cùng trong danh sách đó chỉ thẳng đến nút đầu tiên của danh sách. Cấu trúc của danh sách liên kết vòng kép được mô tả trong hình dưới đây:



Vấn đề quan trọng tiếp theo là cài đặt danh sách liên kết. Để cài đặt được danh sách liên kết, chúng ta phải ít nhất thực được các công việc sau:

- Dùng một cách nào đó để chia bộ nhớ thành các nút, mỗi nút có thành phần dữ liệu và phần liên kết và một cách đặt cho các con trỏ.
- Các thao tác để truy cập tới những giá trị lưu trong mỗi nút.
- Dùng một cách nào đó để ghi dấu các nút đang dùng và các nút tự do, và để truyền các nút giữa hai vùng nói trên.
- **Danh sách liên kết**

  Vấn đề quan trọng tiếp theo là cài đặt danh sách liên kết. Để cài đặt được danh sách liên kết, chúng ta phải ít nhất thực được các công việc sau:

AUM VIỆT NAM

- Dùng một phương tiện nào đó để chia bộ nhớ thành các nút, mỗi nút có thành phần dữ liệu và phần liên kết và một cách đặt cho các con trỏ.
- Các thao tác để truy cập tới những giá trị lưu trong mỗi nút
- Dùng một phương tiện nào để ghi dấu các nút đang dùng và các nút tự do, và để truyền các nút giữa hai vùng nói trên.

Danh sách liên kết không phải là cấu trúc dữ liệu tiền định trong đa số ngôn ngữ lập trình, vì vậy chúng ta phải tiến hành cài đặt danh sách liên kết dựa vào các cấu trúc tiền định khác. Trong danh sách liên kết đơn, mỗi nút gồm hai thành phần: phần dữ liệu để lưu thông tin về một phần tử trong danh sách và phần liên kết chỉ đến nút tiếp theo trong danh sách hoặc là con trỏ rỗng. Cấu trúc của danh sách liên kết được thiết lập bằng các khai báo sau:

```
typedef  Kieu_du_lieu ElementType;
struct NodeType
        {
            ElementType Data;
            struct NodeType *Link;
        }
NodeType *List;
```

Hàm thực hiện thao tác kiểm tra danh sách có rỗng hay không được cài đặt như sau:

```
 int Empty(NodeType *List)
         {
                int empty;
                emty = 0;
                if (List == NULL)
                 emty = 1;
                else
                 emty = 0;
                return emty;
         }
```

Để thực hiện thao tác duyệt các phần tử trên danh sách ta tiến hành cài đặt theo thuật toán dưới đây:

```
void Traverse( NodeType *List)
         {
                NodeType *Currp;
                Currp = List;
                while (Currp!=NULL)
                 {
                    printf("&f", Currp->Data);
                    Currp =Currp->Link;
                 }
         }
```

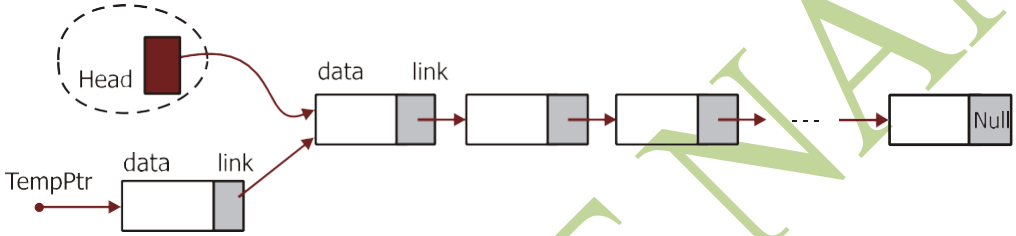
AUM VIỆT NAM

Xây dựng thao tác thêm một nút mới vào danh sách liên kết:

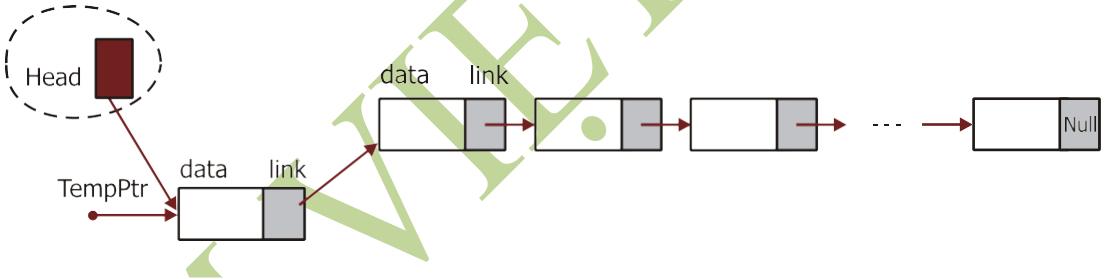
Để chèn được một phần tử vào danh sách liên kết, trước hết ta phải tạo được một nút mới chứa phần tử cần chèn và ta truy cập được tới nút này. Trong thủ tục này nút mới được truy cập bằng con trỏ và dữ liệu của nút này chứa giá trị của phần tử cần chèn. Bước tiếp theo là cần phải xác định xem nút mới này được chèn ở đầu danh sách hay sau một phần tử cho trước trong danh sách.

Nếu là chèn vào đầu danh sách, trước hết ta phải đặt trường liên kết của nút chỏ tới nút đầu tiên của danh sách sau đó đặt lại con trỏ trỏ đến nút đầu tiên trước đây trỏ vào nút mới thêm vào này. Quá trình thêm nút một nút mới vào đầu của một danh sách liên kết được thực hiện như sau:

Bước 1: Đặt `Link(TempPtr` bằng



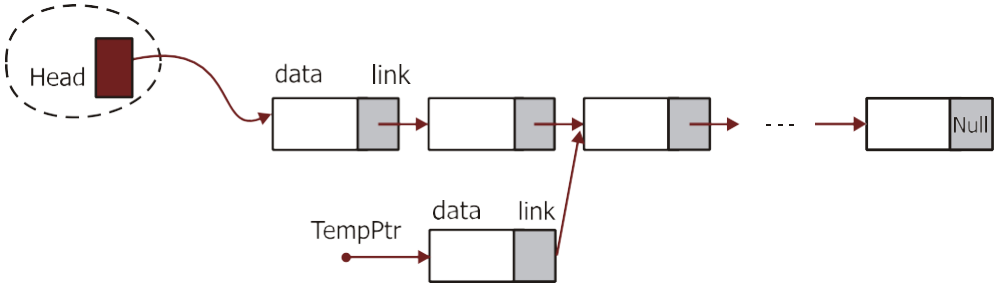
Bước 2: Đặt `List` bằng



Chú ý

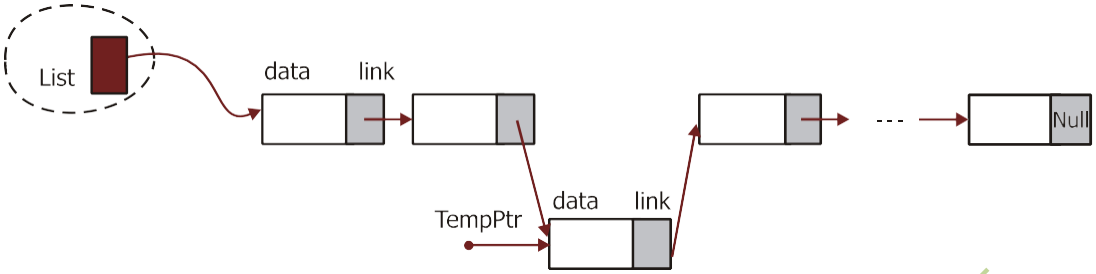
Việc chèn một nút mới vào đầu danh sách phải thực hiện theo đúng thứ tự trên vì nếu ta đặt List bằng TempPtr trước, sau đó mới gắn Link(TempPtr) = List thì sẽ làm cho phần liên kết của nút mới chỉ vào chính nó chứ không chỉ đến nút trong danh sách liên kết, điều này làm ta sẽ không truy cập được đến danh sách ban đầu.

Trong trường hợp thứ hai, khi chèn nút mới vào sau một nút bất kỳ trong danh sách ta đặt con trỏ trong trường liên kết của nút mới trỏ đến nút tiếp theo mà con trỏ của nút tại vị trí chén trỏ tới.



ACVIETNAM

Sau đó đặt con trỏ trong trường liên kết của nút chỉ định trỏ đến nút mới.



Chú ý

Việc chèn này cũng phải được thực hiện theo đúng thứ tự trên để tránh việc mất đi một phần của danh sách.

Từ đây ta có thuật toán thêm một nút mới vào một danh sách liên kết:

Thuật toán này để thêm phân tử Item vào một danh sách liên kết với nút đầu tiên được trỏ bởi con trỏ List tại nút thứ `PrevPtr`. Nếu `PrevPtr = 0`, nút mới được thêm vào đầu danh sách.

```
void Insert(NodeType*List,ElementType Item,NodeType *PrevPtr)
{
        NodeType *TempPtr;
        TempPtr=GetNode(Item);
        //tạo nút mới để chèn vào danh sách
              if (List==NULL)
               {
               List=TempPtr;
               TempPtr->Link=NULL;
               }
              else
               {
               TempPtr->Link=PrevPtr->Link;
               PrevPtr->Link=TempPtr;
               }
}
```

Trong đó thủ tục `GetNode` trả về con trỏ `P` chỉ vào một nút tự do; nếu không còn `P` được trả về giá trị `Null`

```
NodeType* GetNode(ElementType Item)
                  {
                        NodeType *p = new NodeType;
                        p->Data = Item;
                        p->Link = NULL;
                        return p;
                   }
```

AUM VIET NAL

Một trường hợp đặc biệt nếu danh sách liên kết là rỗng thì thao tác chèn thực hiện như trong trường hợp chèn vào đầu danh sách và nút mới được chèn vào là nút duy nhất trong danh sách. Vì `List` có giá trị `null` đối với danh sách rỗng nên sau thao tác đầu tiên sẽ đặt `Link(TempPtr)` bằng `null` và điều này đúng vì nút mới sẽ không có phần tử tiếp theo. Thao tác thứ hai sẽ đặt `List` chỉ vào nút đầu tiên trong danh sách. Trường hợp đặc biệt khác đó là chèn vào cuối một danh sách không rỗng khi đó qua thủ tục trên ta vẫn tiến hành chèn được nút mới vào danh sách. Vì khi đó trường liên kết của nút mới chỉ `null` điều đó báo hiệu nút mới thêm vào sẽ là nút cuối cùng của danh sách mới và trường liên kết của nút cuối cùng trong danh sách cũ sẽ chỉ vào nút mới được thêm vào này.

Đối với thao tác xóa ta cũng phải xét đến hai trường hợp:

- Trường hợp xóa nút đầu tiên trong danh sách liên kết, khi đó ta chỉ cần đặt con trỏ List chỉ vào nút thứ hai trong danh sách sau đó tiến hành xóa nút đầu tiên
- Trường hợp xóa một nút đứng sau nút được chỉ bởi PrevPtr, khi đó ta chỉ cần đặt phần liên kết của nút đứng trước nút định xóa chỉ vào nút sau nút định xóa rồi tiến hành xóa nút muốn xóa.

Thuật toán xóa một nút trong danh sách liên kết:

```
void Deletefromlist(NodeType *List,NodeType *PrevPtr)
    {
          NodeType * TempPtr;
          if (List == NULL)
                    printf("danh sach rong");
          else
              {
                        if  (PrevPtr == NULL)
                                        {
                                                TempPtr = List;
                                                List = TempPtr->Link;
                                        }
                         else
                                        {
                                                 TempPtr = PrevPtr ->
        Link;
                                                 PrevPtr  ->  Link  =
        TempPtr -> Link;
                                        }
                        DeleteNode(List,TempPtr);
               }
             }
```

AUM VIỆT NAM

Trong đó thủ tục `DeleteNode` được cài đặt như sau:

```
void Delete(NodeType *List,NodeType *PrevPtr)
        {
          NodeType *p;
          if (List == NULL)
               printf("danh sach rong");
          else
               {
               p = List;
               while (p -> Link != PrevPtr)
                                              p = p -> Link;
               p -> Link = PrevPtr -> Link;
               free(PrevPtr);
               }
        }
```

Các thao tác chèn và xóa trên của danh sách liên kết được thực hiện trên cơ sở ta đã biết rõ vị trí để chèn vào hay vị trí của nút định xóa. Tuy nhiên trong nhiều trường hợp muốn xóa một nút từ một danh sách liên kết hay chèn một nút mới vào sau một nút nhưng ta chỉ biết trường giá trị của nút đó thì cần phải có phương pháp định vị con trỏ `PrePtr` mà nút đó chỉ tới nút nào trong danh sách. Trong trường hợp này ta có thể sử dụng thuật toán sau để tìm kiếm tuyến tính trong một danh sách liên kết.

Thuật toán tìm kiếm: tìm một nút chứa `Item` trong một danh sách liên kết có nút đầu tiên được chỉ bởi `List`. Nếu tìm ra được nút chứa `Item` thì giá trị `Logic Found` được đặt bằng đúng, con trỏ `CurrPtr` chỉ tới nút đó và con trỏ `PrevPtr` chỉ tới nút đứng sau nó hay `null`. Nếu không có `Found` được đặt giá trị sai.

```
//Bước 1: Đặt
Currptr = List; PrevPtr = Found = 0;
//Bước 2:
while (Found = =0) && (CurrPtr != NULL)
     {
      if  (CurrPtr -> Data = Item)
       Found = 0;
 else
       {
              PrevPtr = CurrPtr;
              CurrPtr = CurrPtr -> Link;
       }
    }
```

AUM VIỆT NAM

Thủ tục cài đặt thuật toán tìm kiếm trên danh sách liên kết được thực hiện như sau:

```
int SearchOfList(NodeType *List,ElementType Item)
  {
        int Found;
        Found=0;
        NodeType *PrevPtr;
        NodeType *CurrPtr;
        CurrPtr = List;
        PrevPtr=NULL;
        while ((Found==0)&&(CurrPtr != NULL))
         {
             if  (CurrPtr->Data ==Item)
              Found=1;
             else
              {
                      PrevPtr = CurrPtr;
                      CurrPtr = CurrPtr->Link;
              }
          }
         return Found;
   }
```

2.4. Ứng dụng danh sách

Danh sách được ứng dụng nhiều trong các cấu trúc dữ liệu mảng: mảng 1 chiều, mảng nhiều chiều; mảng cấp phát tĩnh, mảng cấp phát động; … mà chúng ta đã nghiên cứu và thao tác khá nhiều trên các ngôn ngữ lập trình. Dưới đây là một số ứng dụng của danh sách:

Các phép tính số học trên đa thức

Trong mục này ta sẽ xét các phép tính số học (cộng, trừ, nhân, chia) đa thức một ẩn. Các đa thức một ẩn là các biểu thức có dạng:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0 \qquad (1)$$

Mỗi hạng thức của đa thức được đặc trưng bởi hệ số (coef) và số mũ của x (exp). Giả sử các hạng thức trong đa thức được sắp xếp theo thứ tự giảm dần của số mũ, như trong đa thức (1). Rõ ràng ta có thể nhìn nhận đa thức như một danh sách tuyến tính các hạng thức. Khi ta thực hiện các phép toán trên các đa thức ta sẽ nhận được các đa thức có bậc không thể đoán trước được (bậc của đa thức là số mũ cao nhất của các hạng thức trong đa thức). Ngay cả với các đa thức có bậc xác định thì số các hạng thức của nó cũng biến đổi rất nhiều từ một đa thức này đến một đa thức khác. Để biểu diễn đa thức, trong máy tính, ta có thể chọn hoặc lưu trữ dưới dạng danh sách gồm các phần tử tuyến tính hay bằng danh sách liên kết.

Với cách lưu dưới dạng danh sách tuyến tính, nghĩa là lưu trữ phần thông tin cần thiết ứng với mỗi số hạng của đa thức bởi 1 phần tử của danh sách lưu trữ. Chú ý rằng: mỗi số hạng $a_i x^i$ của đa thức với $n \geq i \geq 0$ ta phải xác định được hệ số a_i và số mũ i.

AUM VIỆT NAM

Nhưng mỗi phần tử của danh sách lưu trữ, thường chỉ ghi nhận 1 giá trị thôi, vì vậy nếu 1 phần tử của danh sách lưu trữ chỉ ghi nhận giá trị của hệ số hệ số a_1 thì số mũ i phải ẩn dụ trong thứ tự của phần tử đó. Và điều đó còn tùy thuộc vào kích thước của danh sách, nếu kích thước của danh sách bằng n + 1 thì ta chỉ có thể lưu trữ được đa thức với số mũ tối đa là n và

V[1] lưu trữ giá trị của a_n

V[2] lưu trữ giá trị của a_{n-1}

.....

V[n] lưu trữ giá trị của a_0

Bất kỳ đa thức nào cũng phải được lưu trữ theo đúng quy ước đó .

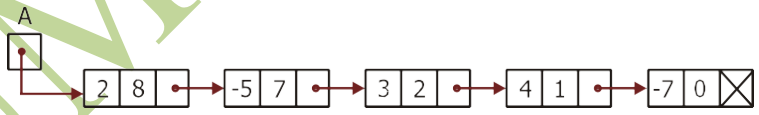
Khi biểu diễn đa thức bằng danh sách tuyến tính ta thấy ưu điểm lớn nhất là các phép toán của đa thức được thực hiện dễ dàng. Nhưng bên cạnh đó cũng xuất hiện 1 nhược điểm rất lớn. Trước hết là kích thước ấn định cho danh sách lưu trữ phụ thuộc vào số mũ lớn nhất của số hạng có trong đa thức. Nếu 1 trong 2 đa thức có mũ cao, nhưng lại ít số hạng, chẳng hạn: $A(x) = 15x^{1000} - x$ thì rõ ràng phải dùng danh sách với kích thước bằng 1001 phần tử để thực hiện lưu trữ. Như vậy thì quá lãng phí bộ nhớ và làm thời gian thực hiện các phép toán diễn ra chậm chạp.

Để khắc phục các nhược điểm của lưu trữ bằng danh sách tuyến tính như đã nêu trên, ta có thể dùng danh sách liên kết để lưu trữ đa thức. Với các lưu trữ này thì đa thức có thể biển diễn dưới dạng danh sách nối đơn mà mỗi nút của nó có quy cách như sau : mỗi nút có 3 trường :

- Trường COEF chứa hệ số khác không của mỗi số hạng trong đa thức.
- Trường EXP chứa số mũ tương ứng.
- Trường LINK chứa địa chỉ nút tiếp theo trong danh sách.

Ví dụ: $A(x) = 2x^8 - 5x^7 + 3x^2 + 4x - 7$

Với đa thức A(x) ở trên thì danh sách biểu diễn có dạng :



Ở đây A là con trỏ, trỏ tới nút đầu tiên của danh sách

Với cách lưu trữ này thì đa thức có bao nhiêu số hạng với hiệu số khác không, danh sách nối đơn biểu diễn nó sẽ có bấy nhiêu nút.

Bây giờ ta xét tới giải thuật thực hiện cộng 2 đa thức A(x) và B(x). Giả sử rằng danh sách biểu diễn chúng đã được tạo lập trong máy và đã được trỏ lần lượt bởi biến trỏ A và B. Ta sẽ gọi C là con trỏ, trỏ tới danh sách biểu diễn đa thức tổng.

Rõ ràng phải dùng 2 biến trỏ p và q để thăm lần lượt các nút, khi duyệt qua 2 danh sách. Khi đó ta thấy có những tình huống như sau :

1) Nếu EXP(p) = EXP(q) ta sẽ phải thực hiện cộng giá trị ở trường COEF của 2 nút đó. Nếu giá trị tổng khác không thì phải tạo ra nút mới để biểu diễn số hạng tương ứng và bổ sung vào (gọi tắt là "gắn vào") danh sách tổng.

AUM VIỆT NAM

2) Nếu EXP (p) >EXP (q), nghĩa là trong danh sách B không có số hạng cùng số mũ với p như trong danh sách A (Ngược lại EXP(q) > EXP (p) thì xử lí cũng tương tự). Như vậy sẽ phải sao chép thông tin ở nút p vào 1 nút mới và gắn vào danh sách tổng.

3) Nếu 1 trong 2 danh sách kết thúc trước thì các nút còn lại của danh sách kia sẽ được sao chép lần lượt vào nút mới và "gắn vào" danh sách tổng. Mỗi lần 1 nút mới được tạo ra thì nó được gắn vào sau nút cuối cùng của danh sách tổng (ta gọi tắt là nút "đuôi" của danh sách tổng). Như vậy, phải thường xuyên nắm được địa chỉ của nút đuôi này, ta gọi đó là d. Ta thấy ngay rằng các công việc :

- "– Xin cấp phát một nút mới
- Sao chép thông tin vào trường COEF và EXP của nút đó.
- "Gắn" nút mới này vào sau nút trỏ bởi d , và lại biến nó thành nút đuôi mới" sẽ được lặp lại nhiều lần trong quá trình xử lí.

Vì vậy ta sẽ viết nó dưới dạng 1 chương trình con và sẽ "gọi nó" khi cần sử dụng.

AUM VIỆT NAM

TÓM LƯỢC CUỐI BÀI

Trong bài này chúng ta đã xem xét đến cấu trúc trừu tượng của danh sách với cách cài đặt quan trọng về danh sách là: mảng và liên kết. Trong cách cài đặt mảng sẽ là rất có ích cho nhiều bài toán xử lý danh sách và trong nhiều ứng dụng nó được ưu tiên hơn cài đặt danh sách. Ví dụ, trong sắp xếp thứ tự và tìm kiếm đòi hỏi truy cập trực tiếp đến mỗi phần tử trong danh sách và điều đó chỉ được thực hiện bằng cài đặt mảng. Cài đặt mảng cũng thích hợp với những danh sách có độ dài cực đại có thể biết trước được và độ dài thực tế không bị thay đổi nhiều trong quá trình xứ lý nhất là với những danh sách ít đòi hỏi các thao tác chèn và xóa hay nếu có những chỉ xẩy ra ở hai đầu của danh sách.

Tuy nhiên, trong cách cài đặt mảng có những điểm yếu đó là chỉ được khai báo dưới một mảng có độ dài chính xác. Hay nói cách khác là độ dài của danh sách sẽ bị giới hạn. Nếu chúng ta không đoán được độ dài lớn nhất của danh sách (mà điều này thường rất khó vì chiều dài của các danh sách thường là "động"). Còn nếu ta dùng mảng có kích thước lớn để lưu danh sách trong khi thực tế danh sách có kích thước rất nhỏ thì sẽ gây lãng phí rất lớn về bộ nhớ. Một điểm yếu khác là việc phải dịch chuyển các phần tử của mảng khi thực hiện thao tác chèn, xóa ở các vị trí không phải đầu cuối danh sách.

Những danh sách có độ dài thay đổi và thường xuyên phải thực hiện thao tác chèn hay xóa đi các phần tử nào đó trong danh sách thì việc cài đặt danh sách liên kết là tốt nhất. Tuy nhiên cách cài đặt này cũng có một số nhược điểm. Một là chỉ có phần tử đầu tiên của danh sách được truy cập trực tiếp, các phần tử khác muốn truy cập được phải đi qua những phần tử đứng trước nó. Hai là phần bộ nhớ bổ sung đòi hỏi trong cài đặt này. Việc cấp phát vùng nhớ không những cho các phần tử của danh sách mà còn cho phần liên kết để nối các phần tử đó. Hơn nữa nếu cài đặt danh sách liên kết trên cơ sở mảng thì chiều dài của danh sách liên kết cũng bị hạn chế. Để khắc phục hạn chế này chúng ta có thể dùng con trỏ để cài đặt cho danh sách liên kết, khi đó kích thước của danh sách liên kết chỉ bị giới hạn bởi bộ nhớ lưu trữ.

AUM VIỆT NAM

BÀI TẬP

1. Dựa vào cài đặt về danh sách liên kết đơn. Hãy dùng mảng để cài đặt cấu trúc dữ liệu của danh sách liên kết đôi.

2. Một danh sách liên kết dùng cài đặt mảng để lưu trữ các ký tự theo sơ đồ dưới đây:

Chỉ số mảng	Data	Link
1	J	**4**
2	z	**7**
3	C	**1**
4	p	**0**
5	B	**3**
6	M	**2**
7	K	**8**
8	Q	**9**
9	?	**10**
10	**?**	**0**

Với List = 5, Free = 6.

a) Viết ra các phần tử của danh sách được sắp xếp theo thứ tự này.

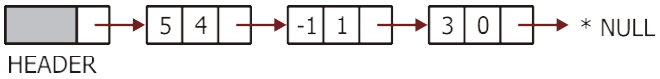
b) Viết ra các nút ở vùng lưu trữ theo thứ tự từ điển của chúng khi được nối với nhau.

3. Viết thủ tục để: nhập vào một dãy các số nguyên nhập từ bàn phím, lưu trữ nó trong danh sách liên kết dạng mảng theo thứ tự nhập vào.

4. Viết thủ tục nhập vào từ bàn phím 1 dãy số nguyên, lưu trữ nó trong một danh sách liên kết có thứ tự không giảm, theo cách sau: Với mỗi phần tử được nhập vào chương trình con phải tìm vị trí thích hợp để xen nó vào danh sách cho đúng thứ tự.

5. Đa thức: $P(x)= a_0 + a_1x + a_2x^2 + \ldots + a_nx^n$, được lưu trữ trong máy tính dưới dạng một danh sách liên kết mà mỗi phần tử của danh sách là một bản ghi có ba trường lưu giữ hệ số, số mũ, và trường NEXT trỏ đến phần tử kế tiếp. Chú ý cách lưu trữ đảm bảo thứ tự giảm dần theo số mũ của từng hạng tử của đa thức.

Ví dụ: đa thức $5 \times 4 - x + 3$ được lưu trữ trong danh sách có 3 phần tử như sau:



a) Viết các khai báo cho cài đặt này của đa thức trên máy tính.

b) Viết thủ tục nhập thông tin của đa thức dựa vào cách lưu trữ này.

c) Viết thủ tục cài đặt phép cộng đa thức.

AUM VIỆT NAM